Search Based Test Generation

EvoSuite

Announcements

- HW6 AFL released
  - Due date

- Major Info Session 4-5pm Thursday

Genetic Algorithms

- Search techniques inspired by evolutionary biology

- Used to find true or approximate solutions to search or optimization problems
  - Chromosome representation
  - Fitness function
  - Selection
  - Crossover
  - Mutation

- What were each of the above for our three examples (meal plan, employee scheduling, test generation)

Project Discussion

Checkpoint 1:

- Due November 15th
- Check in during class November 17th

Checkpoint 2:

- Due November 29th
- Check in during class December 1st

Final Report:

- Due December 18th

Multiple Objective Optimization

Genetic Algorithms with Multiple Objectives

Often times, our fitness function includes multiple objectives which may be conflicting.

- Meal Plan: Maximize nutritional value, minimize cost
- Employee Scheduling: Maximize coverage, minimize overtime
- Test Suite Generation: Maximize branch coverage

```
String example(int a) {
  switch (a) {
     case  0 : return "0";
     case  1 : return "1";
     case -1 : return "-1";
  default: return "default";
}
```

Genetic Algorithms with Multiple Objectives

Fitness functions for multiple objectives:

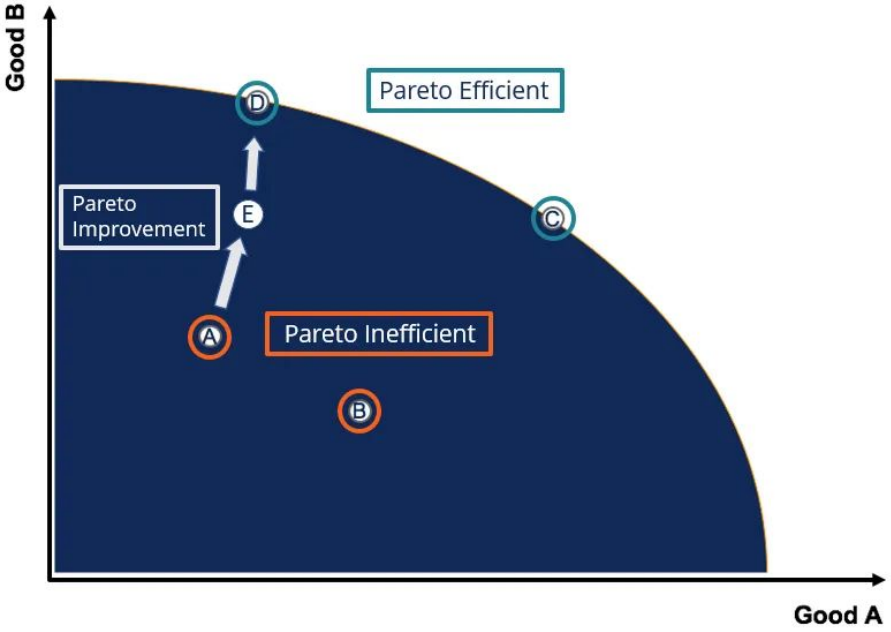
1. Weighted linear combination
    a. Nutritional Value + .5 * cost
    b. This becomes less effective with many goals
    c. Programs often have 100s of branches...

Pareto Fronts

A **Pareto improvement** formalizes the idea of an outcome being "better in every possible way".

A change is called a **Pareto improvement** if it leaves at least one person in society better-off without leaving anyone else worse off than they were before.

A situation is called **Pareto efficient** or **Pareto optimal** if all possible Pareto improvements have already been made; in other words, there are no longer any ways left to make one person better-off, without making some other person worse-off



Pareto Dominance

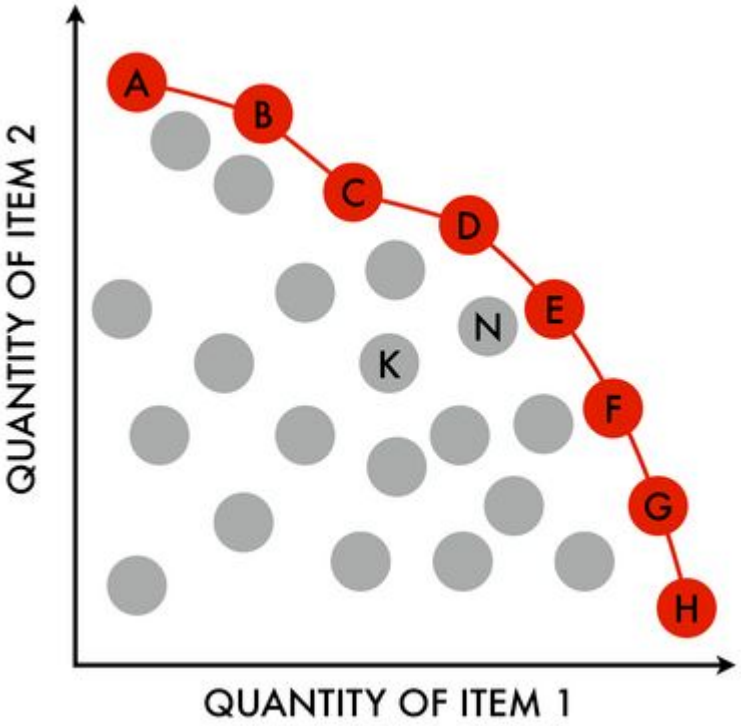
Pareto Dominance: A solution A dominates solution B if A is at least as good as B in all objectives and strictly better in at least one.

Pareto Front: The set of non-dominated solutions representing the trade-offs between conflicting objectives.

Production Possibility Frontier

The red line is an example of a Pareto-efficient frontier, where the frontier and the area left and below it are a continuous set of choices.

The red points on the frontier are examples of Pareto-optimal choices of production. Points off the frontier, such as N and K, are not Pareto-efficient, **since there exist points on the frontier which Pareto-dominate them.**

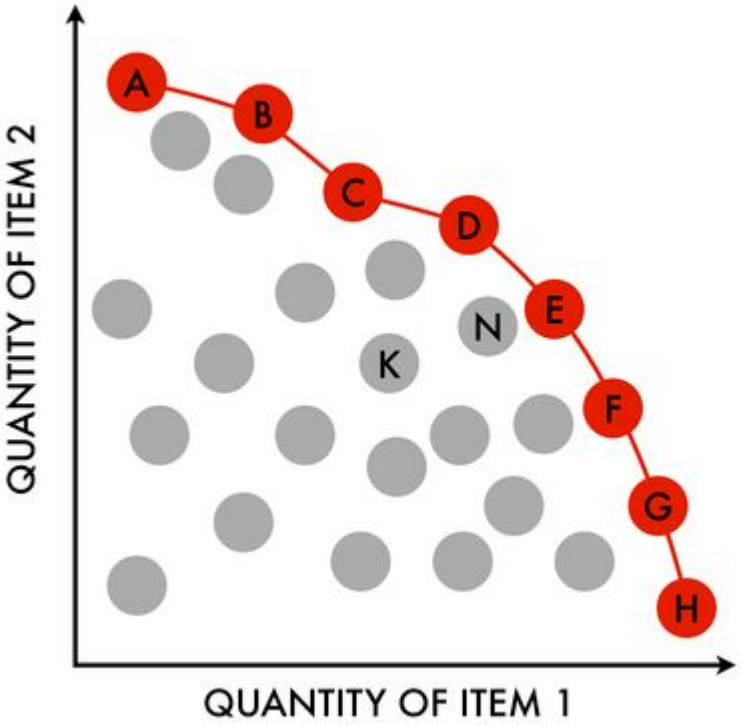


Production Possibility Frontier

Points off the frontier, such as N and K, are not Pareto-efficient, **since there exist points on the frontier which Pareto-dominate them.**

a solution A dominates solution B if A is at least as good as B in all objectives and strictly better in at least one.

What points dominate K and N?



Exercise: Find the Pareto Front

Each test covers the branches listed. Which tests are dominated? Which tests are on the Pareto front?

Test	Branches covered
T1	B1T
T2	B1T, B2T
T3	B1F
T4	B1F, B2F

Back to EvoSuite

EvoSuite uses these concepts in its **selection function**

Calculates pareto optimal solutions

a. Tests which are not “dominated”
b. Dominance is when one test has better fitness on some branch and the other is not better on any other branch
c. If T1 coverages branch 4F, but T2 covers branch 4T, and are the same on all other branches, neither domains the other

Branch Coverage Dominance Example

```
public void unlock(String pin) {
        //check if account holder is admin
        if (pin.equals(decode(admin))) { //B1
                locked = false;
                return;
        }
}
public boolean withdraw(double amount) {
        if (locked) return false; //B2

        if (amount > 0 && balance >= amount) { //B3
                balance -= amount;
                return true;
        }
        return false;
}
```

Dominance is when one test has better fitness on some branch and the other is not better on any other branch

Test Case	Branches Covered
T1	B2 T
T2	B1 F, B2 T
T3	B1 T, B2 F, B3 F

T2 dominates T1

T3 and T2 do not dominate eachother

Making progress on B3 would make progress on B2 F worse

EvoSuite Selection and Fitness

Last class, we simplified fitness: `total branches covered / total branches`

It's actually sorted by dominance ! All tests on the pareto front are equal

Higher branch coverage doesn't mean the test is necessarily better! We want our test suite to include both cases:

- Call to withdraw with locked
- Call to withdraw with unlocked

DynaMosa

We want both T2 and T3 in our test suite, but if we keep B1F as a goal, we will never make progress toward covering B3T

DynaMosa dynamically adapts which goals are included in our ranking

Add T2 to the test suite and remove B1F from our goals

DynaMosa

1. Generate initial population

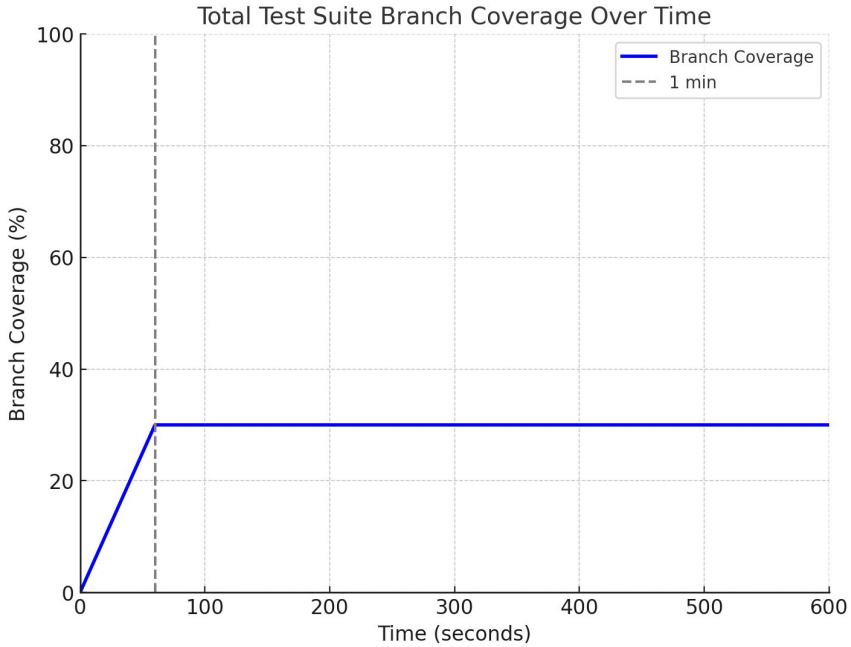
2. Until timeout:
   a. Select Parents
      i. **Requires evaluating population and ranking**
      ii. Rank based selection
   b. Perform Crossover
   c. Perform mutation with some probability
   d. Dynamically adapt goals

3. Output test suite

CodaMosa: Escaping Coverage Plateaus with LLMs

Coverage Plateaus

- SBST works well when mutation / crossover has a non-negligible likelihood of increasing fitness
- Imagine our unlock method...
  - What are the odds we would call unlock with the “magic pin” ?
- Fitness plateaus are common problem in test generation



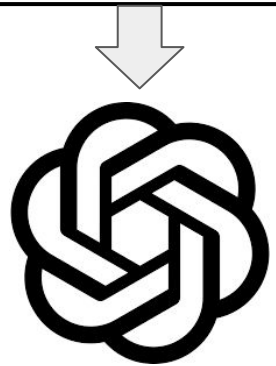
CodaMosa

CodaMOSA, starts SBST and monitors its coverage progress.

When it notices a stall in coverage, it identifies methods that have low coverage

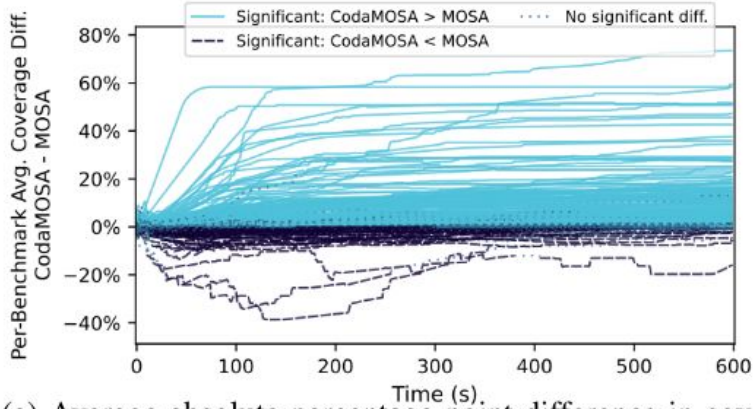
Then, it queries Codex to generate tests for these methods and adds them to the population

```
public void unlock(String pin) {
        //check if account holder is admin
        if (pin.equals(decode(admin))) { //B1
                locked = false;
                return;
        }
}
```

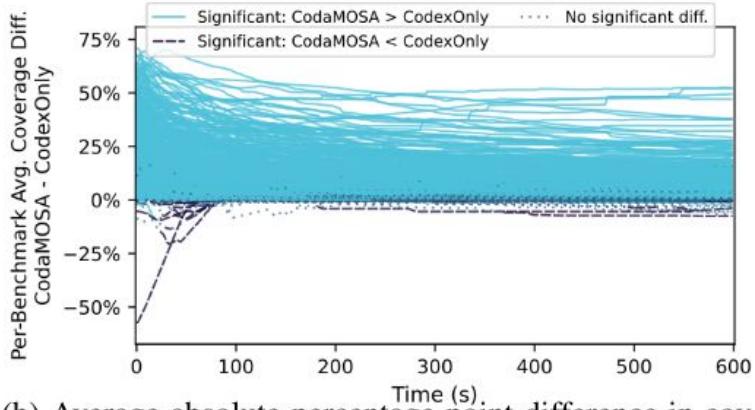


unlock(“admin”)

CodaMosa Evaluation



(a) Average absolute percentage point difference in coverage between CODAMOSA and MOSA.



(b) Average absolute percentage point difference in coverage between CODAMOSA and CodexOnly.

Fuzzing

Fuzzing

- Automated software testing technique
- Involves providing unexpected or invalid inputs and monitoring the programs behavior
- Typically refers to testing C programs



How is this different from randomized testing of Java programs?

```
public class Number {
    private int num;
    public Number(int num) {
        this.num = num;
    }
    public boolean isPrime() {
        if (num <= 1) return 0;

        for (int i = 2; i <= num; i++) {
            if (num % i == 0)
                return false;
        }

        return true;
    }
}
```

```
Scanner scanner = new Scanner(System.in);

int number = scanner.nextInt();

Number  n = new Number(number));

n.isPrime()
```

How is this different from randomized testing of Java programs?

```
int is_prime(int num) {
        if (num <= 1) return 0;
        for (int i = 2; i <= num; i++) {
                if (num % i == 0)
                        return 0;
        }
        return 1;
}
```

```
fgets(input, sizeof(input), stdin);
num = atoi(input);
is_prime(num)
```

The First Fuzzing Study

- Conducted by Barton Miller @ Univ of Wisconsin
- 1990: Command-line fuzzer, testing reliability of UNIX programs
    - Bombards utilities with random data
- 1995: Expanded to network protocols, file systems etc.
- Later: Expanded to GUI-based Windows programs, OS X apps, Android apps, object oriented programs

Fuzzing UNIX Utilities: Aftermath

- 1990: Caused 25-33% of UNIX utility programs to crash (dump state) or hang (loop indefinitely)

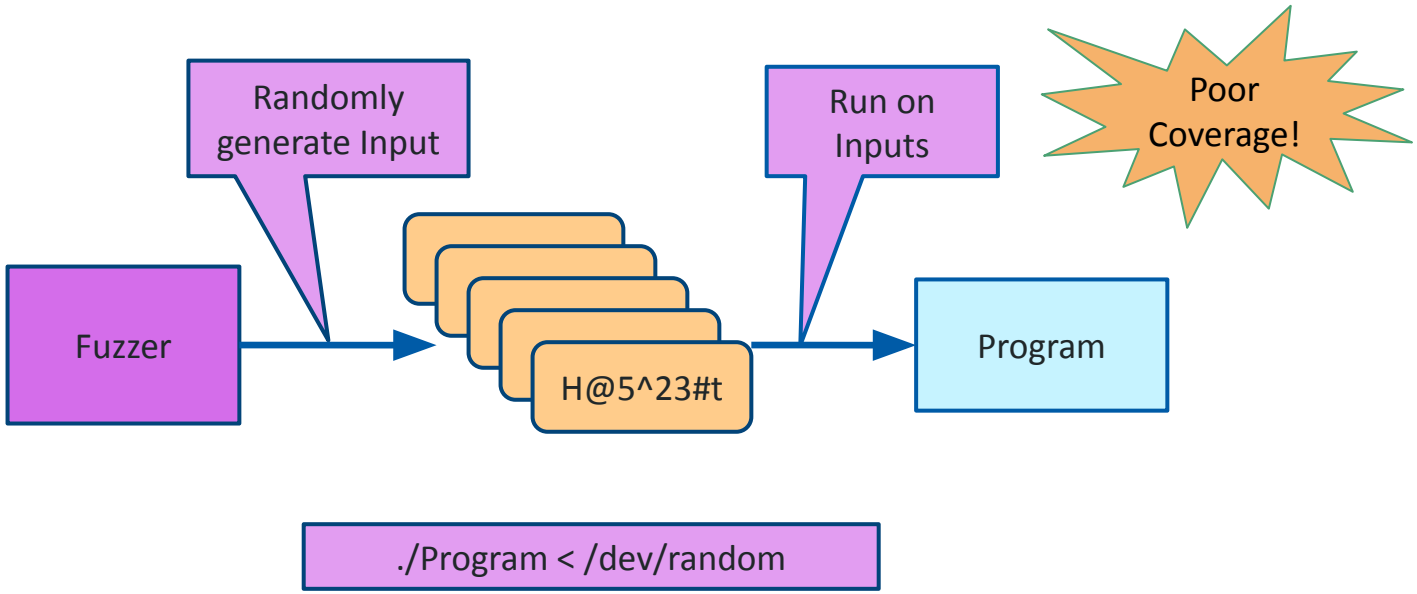
- 1995: Systems got better... but not by much!

  “Even worse is that many of the same bugs that we reported in 1990 are still present in the code releases of 1995.”

3 Generations of Fuzzers

1. Entirely Random
2. Smart Seed Inputs
3. Coverage Feedback

The First Generation



The Second Generation

```
JsonObject parse_json(String input) {

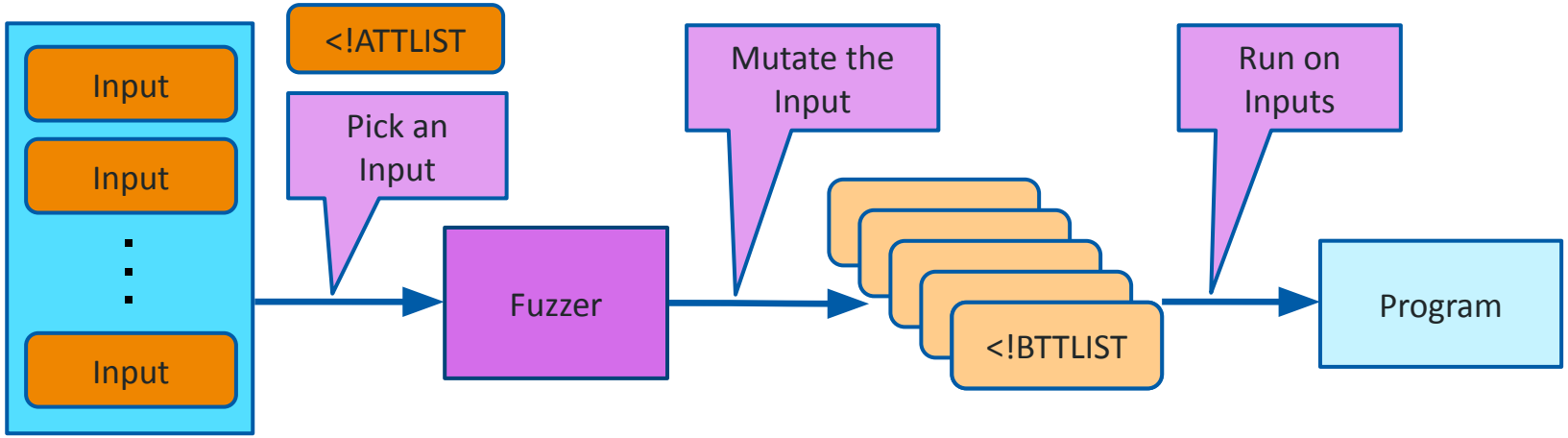
    If (!well_formed(input)) {
        return null;
    }

    if (has_a_list(input)) {
        //BUG
    }

    return JsonObject(input); //exits normally

}
```

The Second Generation



The Second Generation

```
JsonObject parse_json(String input) {

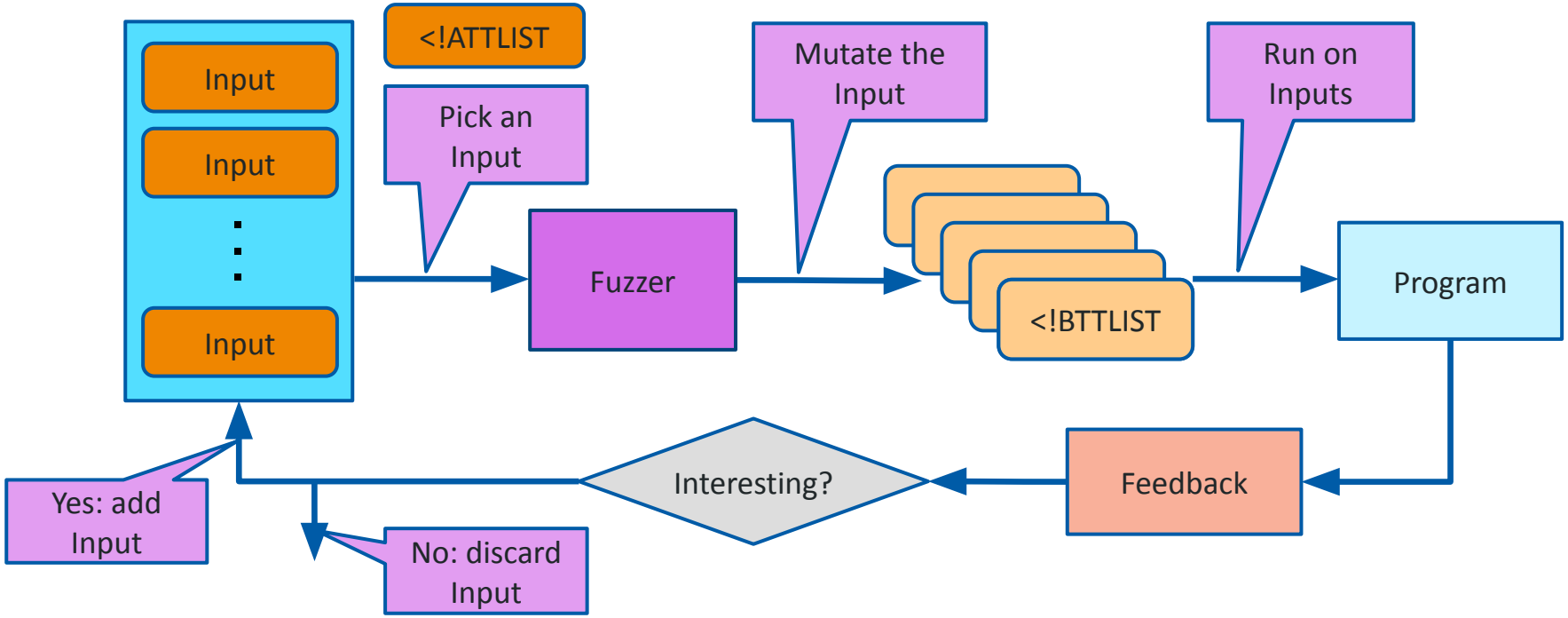
    If (!well_formed(input)) {
        return null;
    }

    if (has_a_list(input)) {
        //BUG
    }

    return JsonObject(input); //exits normally

}
```

The Third Generation



What Kinds of Bugs can Fuzzing Find?

C Programs

- SegFault
- DivByZero
- Other memory safety violations…..
- Assertion violations

java

- Null Pointer Exception
- Divide by Zero
  - Assertion violations
- ...

AFL finds real bugs!

Trophies

- VLC
  - CVE-2019-14437 CVE-2019-14438 CVE-2019-14498 CVE-2019-14533 CVE-2019-14534 CVE-2019-14535 CVE-2019-14776 CVE-2019-14777 CVE-2019-14778 CVE-2019-14779 CVE-2019-14970 by Antonio Morales (GitHub Security Lab)
- Sqlite
  - CVE-2019-16168 by Xingwei Lin (Ant-Financial Light-Year Security Lab)
- Vim
  - CVE-2019-20079 by Dhiraj (blog)
- Pure-FTPd
  - CVE-2019-20176 CVE-2020-9274 CVE-2020-9365 by Antonio Morales (GitHub Security Lab)
- Bftpd
  - CVE-2020-6162 CVE-2020-6835 by Antonio Morales (GitHub Security Lab)
- Tcpdump
  - CVE-2020-8036 by Reza Mirzazade
- ProFTPd
  - CVE-2020-9272 CVE-2020-9273 by Antonio Morales (GitHub Security Lab)
- Gifsicle
  - Issue 130 by Ashish Kunwar
- FFmpeg
  - Ticket 8592 Ticket 8593 Ticket 8594 Ticket 8596 by Andrea Fioraldi
  - Ticket 9099 by Qiuhao Li
- Glibc
  - Bug 25933 by David Mendenhall
- FreeRDP
  - CVE-2020-11095 CVE-2020-11096 CVE-2020-11097 CVE-2020-11098 CVE-2020-11099 CVE-2020-13397 CVE-2020-13398 CVE-2020-4030 CVE-2020-4031 CVE-2020-4032 CVE-2020-4033 by Antonio Morales (GitHub Security Lab)
- GNOME
  - Libxps issue 3 by Qiuhao Li
- QEMU
  - CVE-2020-29129 CVE-2020-29130 by Qiuhao Li
- GNU coreutils
  - Bug 1919775 by Qiuhao Li
- PostgreSQL
  - Crash while parsing zero-symbols in jsonb string by Nikolay Shaplov (Postgres Professional)

Summary

- AFL
  - Coverage guided
  - seed based
  - For C programs

- Multi objective optimization
  - Dynamically adapt goals

- CodaMOSA
  - Query LLM to escape coverage plateaus